

Picking the right chunks

The answer

In a RAG system you hand a model a pile of text chunks and ask it to pick the ones that answer the question. We tested who picks best, and what each picker costs.

Jev (TypeSafe) picked best, finding 24.5% of the right chunks. Jev found 24.5% at \$0.84 per thousand questions — 2.6× cheaper than the next cheapest model in the test and 4× cheaper than deepseek-v4.1-flash.

Every model beat the free option — and by almost exactly the same amount. Before any model is called, a plain keyword search has already put the chunks in some order. All 5 models improved on that order, by between 11.1 and 11.9 percentage points — a spread of 0.8 points across the whole field. So the reranking step is worth paying for, but *which* model does it is not something this test can tell apart. What separates them is price.

A ceiling limits all of this: 33.1%. That is the share of correct chunks the keyword search put in front of the models at all. No picker can find a chunk it was never shown. Every score below sits under that ceiling, and the ceiling is the single biggest lever in the system — bigger than the choice of model.

MODEL	FOUND THE RIGHT CHUNKS	95% RANGE	COST PER 1,000 QUESTIONS	TYPICAL SPEED
Jev (TypeSafe)	24.5%	17.4% to 32.1%	\$0.84	565 ms
glm-5.3-flash	24.4%	17.7% to 31.6%	\$2.19	3,336 ms
deepseek-v4.1-flash	23.8%	17.1% to 31.2%	\$3.68	2,585 ms
qwen3.8-flash	23.6%	16.8% to 30.7%	\$2.92	3,649 ms
gemini-3.8-flash	answered only 81 of 101 — not comparable		\$11.84	3,937 ms
BM25 only (no model)	12.5%	7.0% to 18.6%	\$0.00	0 ms

1 · What we tested

One job: **given a question and 100 text chunks, put the chunks in order, best first.** We then check whether the chunks that actually answer the question ended up in the top 5.

This is the job a RAG system does every time someone asks it something. It is usually called reranking. The 5 best chunks are what gets passed to the model that writes the final answer, so a chunk that does not make the top 5 might as well not exist.

We used 4 real collections, not a made-up test set:

- **BRIGHT** — real questions people posted on StackExchange, each several hundred words long, over a corpus of real documents. It was built specifically so that keyword matching does not work. It is hard on purpose.
- **FiQA** — real finance questions over 218,052 real answer passages. Ordinary difficulty.

Together: **101 questions, 4.1 correct chunks per question on average, 10,100 chunks ranked per model.**

Nobody wrote these questions or judged these answers for us. The questions are real and the judgments of which chunk is correct come with the datasets.

2 · How we tested it

Every model saw exactly the same thing. Same questions, same 100 chunks, in the same order, asked the same way. The only thing that changed was the model.

The 100 chunks are not random. They come from a keyword search (BM25) run over the whole corpus — the same first step a real RAG system uses. So the wrong chunks are genuinely tempting wrong chunks: text that mentions the right words but does not answer the question. That is what makes the job hard.

The two kinds of model answer differently, so each was asked in its own native way:

- **Jev** is given the chunks as its options and returns a score for every one. That ordering is the answer. There is no prompt to tune and no output format to enforce.
- **Chat models** are given the chunks in a prompt and asked to reply with a JSON list of the best 10 chunk ids.

We also ran a **free baseline**: the keyword search's own order, with no model involved. This matters more than it sounds. It is the thing every model has to beat to justify existing.

What we count. For each question, what share of the correct chunks made the top 5. Averaged over all 101 questions. We report a 95% range beside every score, worked out by resampling the questions — so you can see when two models are too close to separate.

3 · The analysis

MODEL	BRIGHT BIOLOGY	BRIGHT ECONOMICS	BRIGHT PSYCHOLOGY	FIQA FINANCE
Jev (TypeSafe)	22.3%	12.2%	8.0%	34.7%
glm-5.3-flash	27.0%	6.8%	9.4%	33.3%
deepseek-v4.1-flash	27.8%	10.2%	8.9%	32.0%
qwen3.8-flash	25.8%	11.5%	8.9%	32.0%
gemini-3.8-flash	27.8%	11.5%	2.7%	34.9%
BM25 only (no model)	7.8%	2.0%	1.2%	21.6%

The two collections are different problems. FiQA scores are several times higher than BRIGHT scores for every model, including the free baseline. BRIGHT was designed to defeat keyword search, and it does: the keyword step hands over very little that is correct, so there is little for any picker to find.

Where the models help is not where you would guess. The lift each model gives over the free keyword baseline, measured question by question on the same questions:

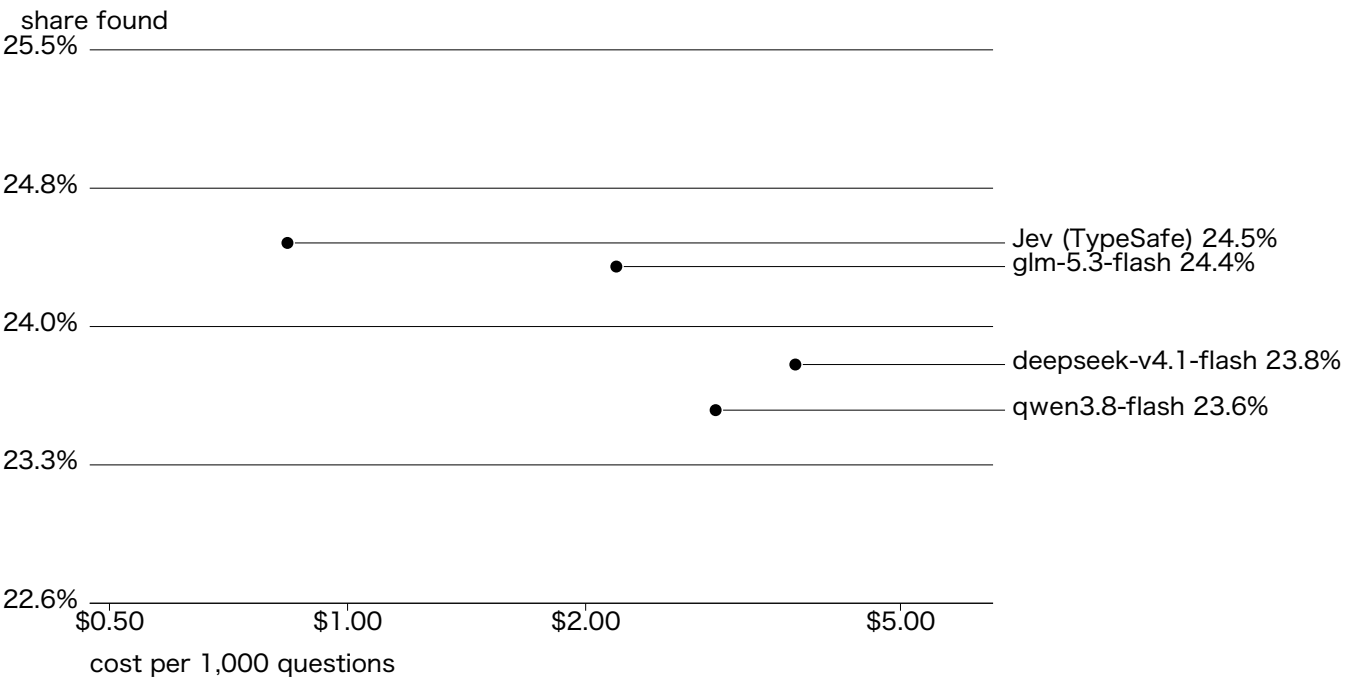
MODEL	BETTER THAN FREE KEYWORD SEARCH BY	95% RANGE	VERDICT
Jev (TypeSafe)	+11.8%	6.1% to 18.0%	really helps
glm-5.3-flash	+11.2%	5.9% to 16.9%	really helps
deepseek-v4.1-flash	+11.2%	5.4% to 17.4%	really helps
qwen3.8-flash	+11.1%	5.6% to 17.1%	really helps
gemini-3.8-flash	+11.9%	6.3% to 18.0%	really helps

A "tie" verdict means the test cannot tell that model apart from doing nothing. It does not mean the model is bad. It means **this sample of 101 questions is not enough to prove it helps**, and paying for it is a bet rather than a decision.

	1	2	3	4	5	6
1 Jev (TypeSafe)	-	=	=	=	=	+
2 glm-5.3-flash	=	-	=	=	=	+
3 deepseek-v4.1-flash	=	=	-	=	=	+
4 qwen3.8-flash	=	=	=	-	=	+
5 gemini-3.8-flash	=	=	=	=	-	+
6 BM25 only (no model)	-	-	-	-	-	-

A + means the model on that row really is better than the model in that column. A = means we cannot tell them apart. Most of this grid is = , and that is the honest result.

4 · Cost against effectiveness



The horizontal axis is cost per thousand questions and it is a log scale — each step is ten times the last. It has to be, because the cheapest and dearest models here are 4× apart in price while sitting within about a point of each other on quality.

Jev sits at the far left. Its price comes almost entirely from reading the chunks, because it charges nothing for what it writes back. A reranking job is nearly all reading: 100 chunks in, a short list out. That is the single reason the gap is this wide, and it would narrow on a job that produced long output.

The premium model could not be run at this size. deepseek-v4.1-flash was the dearest model we could afford across all 101 questions. Running sonnet-5 over the same 100 chunks would have cost \$24.73 — more than this entire experiment. That is a real constraint, not a rhetorical one, and it is why the premium comparison below was run on a shorter list.

The same test with only 20 chunks

We ran a second, smaller version: the same questions, but only the top 20 chunks, which is cheap enough to include a premium model.

Jev scored 17.9%. Total spend \$0.58.

sonnet-5 could not be measured here. Only 37 of 101 of its calls completed — the rest were refused by the provider for want of prepaid credit, not by the model. The questions that did get through are the ones that happened to run first, not a fair sample, so averaging them would produce a number that looks like a result and is not one. It is left out rather than dressed up.

The 20-chunk version is much harder than it looks, because with only 20 chunks the keyword step hands over almost nothing correct on the BRIGHT questions. The lesson is not about the models. It is that **giving the picker more chunks to look at raises the ceiling faster than upgrading the picker does.**

5 · What could be wrong with this

The ceiling dominates everything. At 33.1%, most correct chunks were never shown to any model. A better first-stage retriever — a proper embedding search instead of keyword matching — would change every number here, probably by more than swapping models does. We used keyword search because it is reproducible with no dependencies and no training, not because it is the best available.

101 questions is a small sample. That is why most comparisons come back as ties. The ranges in the tables are real; if two of them overlap, we are not claiming a winner.

One run, no repeats. Every model was asked once per question at temperature 0. We did not measure run-to-run variation.

The judgments are binary and incomplete. A chunk is either marked correct or it is not. A genuinely useful chunk nobody happened to judge counts as wrong, for every model equally.

Prompt shape favours nobody deliberately, but it is one shape. The chat models got one prompt. A different prompt, or a purpose-built reranking model, could do better. We did not tune per model, because tuning one and not the others is how benchmarks get rigged.

We probably asked Jev the wrong way, and that works against Jev. Jev offers three ways to ask a question: *choice* (pick one from a list), *score* (rate it on a scale) and *noul* (is this true?). We used **choice**, with all 100 chunks as the options in a single question — so the chunks compete for one pool of probability. Since publishing, we checked what other people building rerankers on Jev actually do, and neither of the two public ones does it our way: one asks a separate *noul* question per chunk — "is this chunk relevant?" — and sorts by the answer; the other uses a *score* rubric, one chunk at a time. Both give each chunk an independent judgement instead of making them compete. That is the shape the protocol is documented for.

This was an encoding mistake, not a fault in the product. It ran correctly — every call returned a full set of scores, none were dropped, and none hit the size limit — but the number it produced is **most likely a floor for Jev rather than its best**. Jev already came first on accuracy here while being the cheapest, so the mistake did not change who won; it may have understated by how much. Re-running with *noul* would cost about \$0.09 and is the first thing to do if this test is repeated.

Cost is what the provider billed us, read back from each call, not estimated from a price list.

Appendix A · The protocol in full

- **Questions:** 101 (BRIGHT biology, BRIGHT economics, BRIGHT psychology, FiQA finance).
- **Chunks per question:** 100, from BM25 over the full corpus.
- **Asked for:** the best 10 chunk ids, best first.
- **Scored on:** share of correct chunks in the top 5.
- **Models:** 5, plus the free keyword baseline.
- **Calls:** 505. **Total spend:** \$2.17.
- **Temperature:** 0. Reasoning disabled where the provider allows it.
- **Seed:** fixed, so every number in this report reproduces exactly.

What would make this test unfair to the other models

We checked these deliberately:

- *Jev sees the chunks in a better order.* It does not. Every model gets the identical BM25 order.
 - *Jev is asked an easier question.* It is asked to score all 100; the chat models are asked for 10. If anything that is more work, not less.
 - *Ties are broken in Jev's favour.* They are broken by the original retrieval position, identically for every model.
 - *Failed calls are scored as zero.* They are not. They are excluded and counted separately, in Appendix C.
-

Appendix B · The full measurement table

MODEL	QUERIES	@1	@3	@5	@10	OF WHAT WAS FINDABLE	NDCG@5	MRR	INPUT TOKENS	COST	P50	P95
Jev (TypeSafe)	100	13.8%	22.1%	24.5%	26.4%	72.2%	25.3%	33.0%	2,019,084	\$0.085	565	685
glm-5.3-flash	96	16.1%	20.7%	24.4%	29.2%	75.7%	26.7%	34.9%	1,750,770	\$0.221	3,336	12,517
deepseek-v4.1-flash	100	16.5%	20.7%	23.8%	28.2%	73.3%	26.6%	34.7%	1,828,033	\$0.372	2,585	8,943
qwen3.8-flash	101	16.1%	21.0%	23.6%	28.1%	73.3%	26.1%	34.0%	1,920,147	\$0.295	3,649	5,884
gemini-3.8-flash	81	16.2%	19.3%	22.2%	25.1%	76.4%	25.0%	33.3%	1,550,923	\$1.196	3,937	16,041
BM25 only (no model)	101	5.4%	8.5%	12.5%	16.0%	33.4%	11.1%	16.7%	0	\$0.000	0	0

"Of what was findable" is the score counting only the questions where a correct chunk was actually in the 100 shown — the picker's own share of the work, with the retriever's misses removed.

Appendix C · Failures and invented chunks

MODEL	CALLS THAT FAILED	WHY	QUERIES WHERE IT INVENTED A PASSAGE ID	INVENTED IDS IN TOTAL
gemini-3.8-flash	20	http_error, malformed	0	0
glm-5.3-flash	5	bad_ids, malformed	1	2
deepseek-v4.1-flash	1	bad_ids	3	3
Jev (TypeSafe)	1	http_error	0	0

Inventing a chunk id is not the same as ranking badly. A model that returns an id that does not exist has failed in a way a low score would hide, so it is counted on its own. deepseek-v4.1-flash did this most, on 3 questions.

Across the whole run there were 27 failed calls.

Some failures were ours, not the model's. gemini-3.8-flash hit HTTP 402 — our own prepaid balance had too little headroom left for the provider to accept the request. Those calls are counted as failures for honesty, but they say nothing about the model.

Appendix D · How the ranges were worked out

A score here is an average of per-question fractions, not a pass/fail rate. So the interval is not the one used in the ticket-routing report.

Instead: resample the 101 questions with replacement, recompute the average, repeat 4,000 times, and read off the 2.5th and 97.5th percentiles. This assumes nothing about the shape of the scores.

Comparisons between two models are **paired**: both answered the same questions, so the difference is taken question by question before resampling. Unpaired, the variation between easy and hard questions would swamp every real difference and the whole grid would read as ties.